

```
from google.colab import files
uploaded = files.upload()
```

Screenshot... 133747.png

Screenshot 2026-08-07 133747.png(image/png) - 142757 bytes, last modified: 8/7/2026 - 100% done
Saving Screenshot 2026-08-07 133747.png to Screenshot 2026-08-07 133747.png

◆ Gemini

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

# Get the name of the uploaded file
image_filename = list(uploaded.keys())[0]

# Read the image data from the uploaded dictionary
image_data = uploaded[image_filename]
image = cv2.imdecode(np.frombuffer(image_data, np.uint8), cv2.IMREAD_COLOR)

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Sobel filters to get gradients in x and y directions
gradient_x = cv2.Sobel(gray, ddepth=cv2.CV_64F, dx=1, dy=0, ksize=3)
gradient_y = cv2.Sobel(gray, ddepth=cv2.CV_64F, dx=0, dy=1, ksize=3)

# Compute the magnitude of the gradient (or combine them as per original intent)
# If the intent was to subtract to get some form of edge difference, ensure proper scaling
# For general sharpening, magnitude is often used.
# Let's assume for now the intent is a sharpened version, maybe not just difference.
# A common approach for sharpening with Sobel is to find the magnitude and add it back.
# For the original gradient = cv2.subtract(gradient_x, gradient_y):
# This directly calculates the difference between x and y gradients.
# The result will have negative values, so it needs normalization.
gradient = cv2.subtract(gradient_x, gradient_y)
```

```
# Normalize the gradient image for display/saving (scale to 0-255)
# First, take the absolute value as differences can be negative
abs_gradient = cv2.convertScaleAbs(gradient)

# Display original and processed image
cv2.imshow('image', image)
cv2.imshow('abs_gradient', abs_gradient)

# Save the normalized gradient image
cv2.imwrite('sharpened_image3.jpg', abs_gradient)
```



